The turtle graphics module

This allows us to make a simple game

Graphics in Codespaces

- Codespaces by default has no where to to put any graphics
- But in the template repository I have set up a virtual desktop
- After starting the Codespace, go to Ports, and right-click / Ctrl-click on port 6080 to “Open in Browser”

The turtle module

- Inspired by the LOGO programming language from the 60s
- turtle.Screen() creates a window and returns python data representing that window
- turtle.Turtle() creates a small object inside the window and returns python data representing that object

Appearance of the turtle object

- turtle.shape(<name>)
  - <name> can be “arrow”, “turtle”, “circle”, “square”, “triangle”, or “classic”.
- turtle.shapesize(<num>)
  - stretches the shape by <num> times
- turtle.color(<color>)
  - <color> can be a string in https://www.tcl-lang.org/man/tcl8.4/TkCmd/colors.htm or a hex code string “#abcdef”
- Also can use turtle.color(<numR>, <numG>, <numB>)

Event-driven programming

- In event-driven programming, we want the flow of execution to be determined by external events, such as clicking on the turtle
- We have already seen this with the input() function
- We use turtle.onclick(<func>) to specify what function will be executed when there is a click.
- turtle.onclick() requires <func> to have two parameters: the x and y coordinates of the click
- turtle.screen.mainloop() waits for an external event (in a near-infinite loop!) and starts the specified function
- Without mainloop() when the python script reaches the end of the file, the window would close and there would be not way to get input from the user
- Try making the turtle change colors when it is clicked

Turtle movement

- turtle.right(<angle>) and turtle.left(<angle>)
  - Turn the turtle by <angle> degrees
- turtle.forward(<num>) and turtle.backward(<num>)
  - Move the turtle <num> pixels in direction it is facing, or opposite
- turtle.goto(<x>, <y>) and turtle.teleport(<x>, <y>)
  - Turtle moves to the coordinates specified by <x>, <y> (but goto does it slowly)
- Turtle.home()
  - Turtle moves to (0, 0) which is the center of the screen

Make some changes

- We can go to a random position on the screen when it is clicked
  - turtle.screen.window_width() and turtle.screen.window_height() will be useful, and remember that the center of the screen is (0, 0)
- We can make the turtle go to a random position every second … but be wary of making it so we can't reach mainloop()
- We can use turtle.screen.update() instead
- import time and time.sleep() will be needed
- Or use turtle.ontimer(<func>, <t>) - with <t> in milliseconds